

Mandatory README Requirements

1. Problem Understanding

- What problem are you solving?
- Why is this system needed?

2. Architecture Overview

- High-level system architecture
- Media flow (frontend → backend → storage → transcription)
- WebSocket/event flow explanation

3. Technical Decisions & Tradeoffs

- Explain: Why you chose your approach
- Explain: Why streaming over full upload
- Explain: Why your chosen architecture/design

4. Failure Scenarios & Edge Cases

- Mention: Network interruptions
- Mention: Duplicate chunks
- Mention: Camera/mic disconnects
- Mention: Partial upload failures
- Mention: WebSocket reconnects
- Mention: Empty/corrupted media chunks

5. Recovery Mechanisms

- Explain: How your system handles reconnects
- Explain: Retry/recovery logic
- Explain: Chunk recovery strategy
- Explain: Failure handling approach

6. Product Thinking

- Explain: Recruiter experience considerations
- Explain: Candidate experience considerations
- Explain: How suspicious activities are tracked
- Explain: UX decisions made

7. Scalability Considerations

- Mention: What may break at scale
- Mention: Performance bottlenecks
- Mention: Future improvements for high concurrency

8. Observability & Debugging

- Explain: Logging strategy
- Explain: Error tracking
- Explain: How production failures can be debugged

9. AI Usage Documentation

- Clearly document: How you used AI tools
- Clearly document: What prompts/thought process you used
- Clearly document: What decisions were yours vs AI-assisted

10. Demo & Walkthrough

- Include: Setup instructions
- Include: Demo video
- Include: Live link (if available)
- Include: System walkthrough explanation

Task 2: AI Video Interview System

This documentation outlines the technical architecture, design decisions, and implementation details for the AI-driven automated video interview platform.

1. Problem Understanding

The primary goal is to automate the first round of candidate screening by providing an "AI" interviewer that can verbally ask questions and capture candidate responses in real-time.

- **Problem:** Manual first-round interviews are time-consuming and difficult to scale.
- **Need:** Recruiters need a way to screen hundreds of candidates asynchronously while maintaining a high-fidelity record of candidate responses, technical ability, and communication skills.

2. Architecture Overview

The system utilizes a modern MERN stack with a microservices-inspired asynchronous processing layer.

High-Level System Architecture

The architecture is divided into a React/Next.js frontend and a Node.js/Express backend, with Python-based AWS Lambda workers or you can use [next.js](#) too handling heavy media processing.

Media Flow

1. **Frontend:** Uses `MediaRecorder` API to capture video/audio in small Blobs (chunks).
2. **Streaming:** Chunks are sent via WebSockets or high-frequency POST requests to the backend.
3. **Storage:** Raw chunks are stored in Cloudflare R2/AWS S3.
4. **Processing:** An asynchronous queue (`AUDIO_MERGE_QUEUE_URL`) triggers FFmpeg to merge chunks.
5. **Transcription:** The final merged file is sent to Deepgram for Speech-to-Text conversion.

WebSocket/Event Flow

Real-time proctoring and state updates are managed via WebSockets to detect "suspicious" behaviors like tab switching or face absence.

3. Technical Decisions & Tradeoffs

- **Streaming over Full Upload:** We chose streaming to ensure that if a candidate's session disconnects, we have already captured 90% of the interview. This also prevents large, memory-intensive file uploads at the end of the session.
- **Asynchronous AI Processing:** Heavy tasks like transcription and evaluation are handled via SQS queues and Lambda to keep the main API responsive and minimize question transition latency (< 2 seconds).

4. Failure Scenarios & Edge Cases

Scenario	Risk
Network Interruptions	Partial data loss or disconnected WebSocket.
Corrupted Chunks	Media reconstruction failure.
Camera Disconnect	Loss of video feed during the response.
Empty Media Chunks	Storage bloat with non-functional data.

5. Recovery Mechanisms

- **State Persistence:** Session data is stored in `AiInterview.session_data` (central brain), allowing candidates to resume exactly where they left off.
- **Chunk Recovery:** Backend logic ensures chunks are written with deterministic keys (`chunk_NNN.wav`) so FFmpeg can re-order them correctly even if they arrive out of sequence.
- **Socket Reconnection:** Automatic retry logic for WebSocket events to maintain proctoring streams.

6. Product Thinking

- **Candidate UX:** A mandatory "Hardware Check" page ensures the camera and microphone are functional before the interview starts, reducing support tickets.
- **Recruiter UX:** The dashboard allows for "drill down" views where recruiters can see the resume, transcript, and video playback in a single unified view.
- **Suspicious Activity:** Real-time flagging of tab-switching and "face absence" provides recruiters with confidence in the integrity of the interview.

7. Scalability Considerations

- **Queue Bottlenecks:** At scale, the `TRANSCRIPTION_QUEUE` could experience delays. We address this by using serverless Lambda functions that scale horizontally with load.
- **Concurrent Recording:** Large numbers of simultaneous video streams require robust storage ingress (R2/S3) and high-concurrency SQS workers.

8. Observability & Debugging

- **Logging:** We use AWS CloudWatch to track application events and SQS processing times.
- **Error Tracking:** Failed chunks or transcription errors trigger alerts in the "Evaluation Pipeline" for developer review.

9. AI Usage Documentation

- **Thinking Accelerator:** AI was used to structure the initial SQS queue architecture and plan the FFmpeg merge logic.
- **Prompt Strategy:** Used "Understand → Explore → Decide" prompts to validate technical tradeoffs between different TTS providers.
- **Human Oversight:** Every line of code generated for the "Evaluation Orchestrator" was manually reviewed to ensure technical scoring accuracy.

10. Demo & Walkthrough

- **Setup Instructions:**  File
- **Walkthrough Video:**  File
- **Live Demo Link:**  Place
- **Submission Date:**  Date

Submission

- **Video Walkthrough (Required):** Record a video walking through your solution. Cover how you approached the problem, key architectural decisions you made, and — importantly — how you used AI tools in your workflow. This includes what you prompted, where AI accelerated your thinking, and where you had to reason through things yourself. The video should make it clear you understand every part of what you built.
- **Live URL (Required):** If you have deployed the application, share a live link. Candidates who provide a working hosted demo will be evaluated more favorably.